

Binary Tree Traversals

TIP101

Agenda

1 Welcome 🙌

2 Searching and Traversing

3 **BREAK**

4 Collaborative Problem Solving

5 Wrap Up

10 minutes

35 minutes

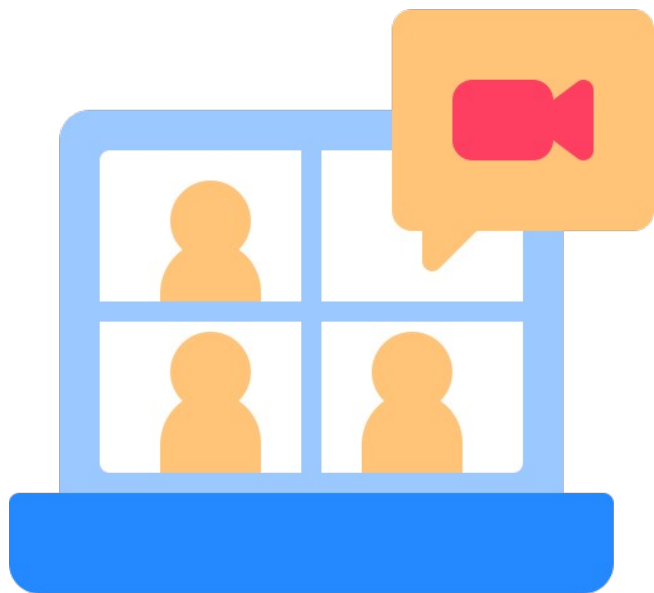
5 minutes

60 minutes

10 minutes

10 minutes

Welcome! 🖐️



**Turn on your
cameras**



**Fill out the entry
ticket**



Icebreaker

5 minutes



Question:

A CS or similar degree can be used for many non-engineering roles. Can you guys identify any long term roles that you could see yourself in, if it wasn't exactly software engineering but still related to tech?



Answer:

Share your answer in the Zoom chat below. What kinds of pets do you have if any? If you don't have any, share what type you would like to have, if any.

Your Feedback



Glows

- + Amazing feedback and gratitude for our volunteers!
- + Initial lessons on Trees are landing well.



Grows

- Course portal lacking tree starter code for certain breakout problems.
- Tree practice(will see more code today)



Announcements & Reminders

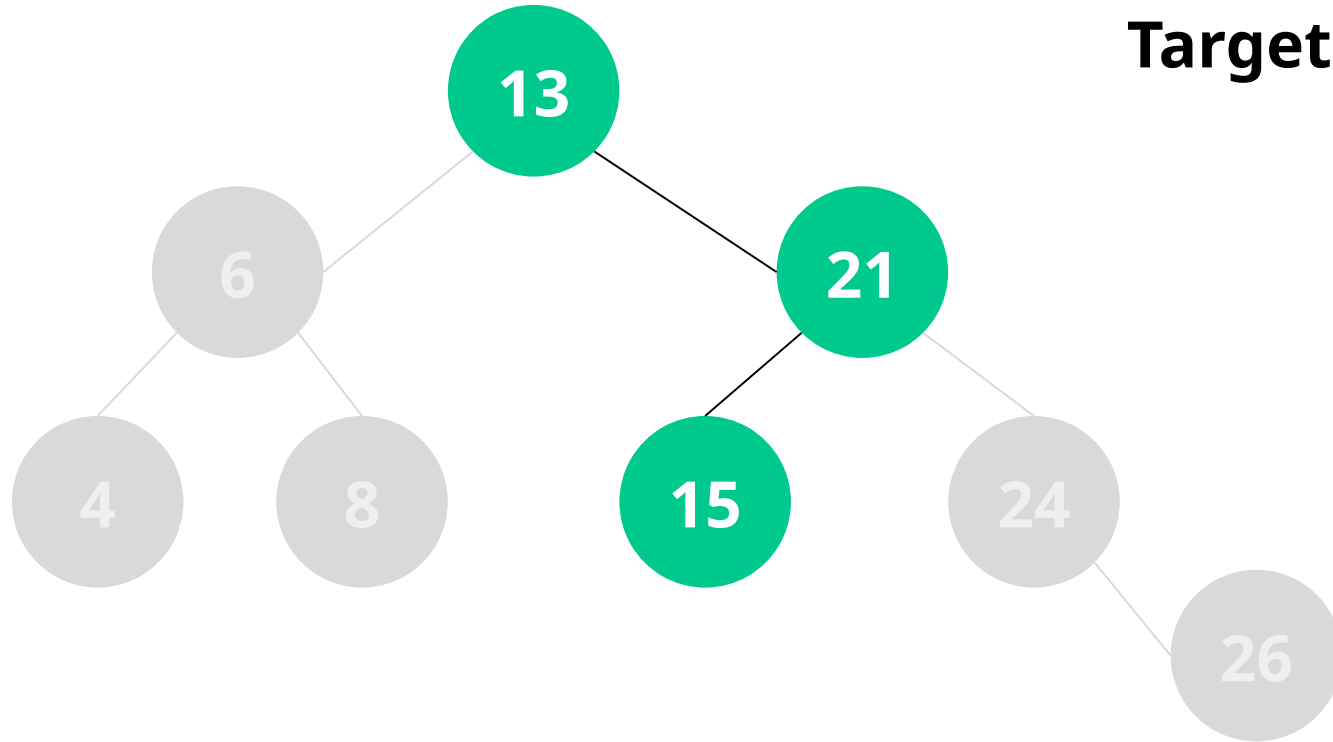
- * Reminder to make sure any open questions or concerns are addressed in these next two weeks!
- * Peer to peer interviews on Thursday!

35 minutes

Searching and Traversing

Node Searching

Target: 15



Node Searching Algorithm

1. Start at the **root**.
2. Compare the target value with the current node's value.
3. If the target value is equal to the current node's value, the node is found.
4. If the target value is less than the current node's value, move to the left child.
5. If the target value is greater than the current node's value, move to the right child.
6. Repeat steps 2-5 until the target value is found or the subtree is empty.

Node Searching Algorithm

1. Start at the **root**.
2. Compare the target value with the current node's value.
3. If the target value is equal to the current node's value, the node is found.
4. If the target value is less than the current node's value, move to the left child.
5. If the target value is greater than the current node's value, move to the right child.
6. Repeat steps 2-5 until the target value is found or the subtree is empty.
 - a. **Match: Repeat → Recursion!**



Check for Understanding

How does the structure of a BST make searching efficient?

- A. It allows searching only the left subtree for all values.
- B. It ensures that nodes are added in a balanced manner.
- C. It allows you to decide whether to go left or right based on the value you're searching for, reducing the number of nodes to check.
- D. It keeps all nodes at the same depth, making the search process uniform.



Idea Reminder: Recursive Concepts

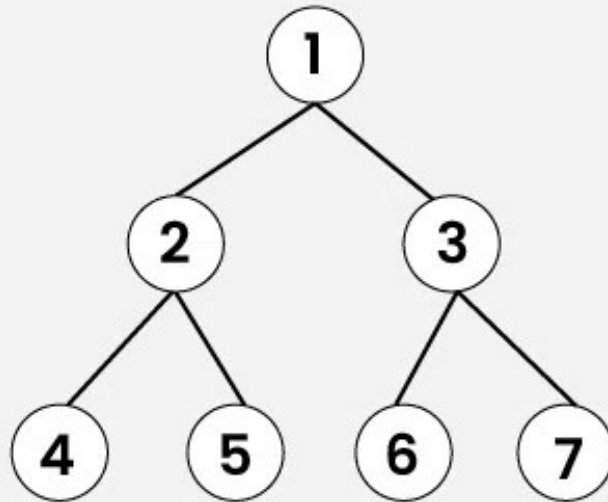
- ☐ Since we use recursion for tree traversal, we will use the word “**root**” frequently when discussing techniques.
- ☐ At every recursive step of traversal, “root” essentially is looking at a different node. This is the idea of **subtrees**.
- ☐ **All recursive** functions are made up of the same two parts:
 - Guesses?



Idea Reminder: Recursive Concepts

- Since we use recursion for tree traversal, we will use the word “**root**” frequently when discussing techniques.
- At every recursive step of traversal, “root” essentially is looking at a different node. This is the idea of **subtrees**.
- **All recursive** functions are made up of the same two parts:
 - Base case & Recursive step(s)
 - Today, we will see that we sometimes have multiple recursive steps. Mainly, when we have **no more nodes** to traverse.
 - For trees we will focus on small subtrees then expand the concepts out.

Tree Traversal



Inorder Traversal

4	2	5	1	6	3	7
---	---	---	---	---	---	---

Preorder Traversal

1	2	4	5	3	6	7
---	---	---	---	---	---	---

Postorder Traversal

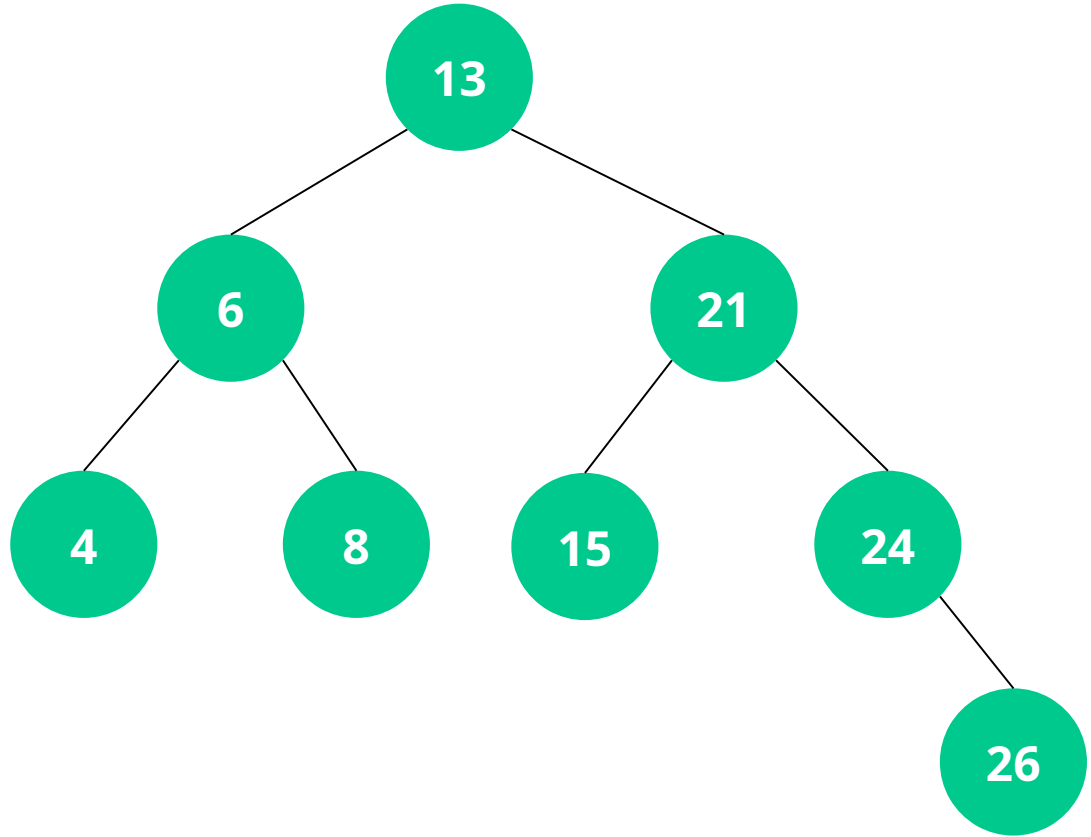
4	5	2	6	7	3	1
---	---	---	---	---	---	---

Level order Traversal

1	2	3	4	5	6	7
---	---	---	---	---	---	---

In-order Traversal

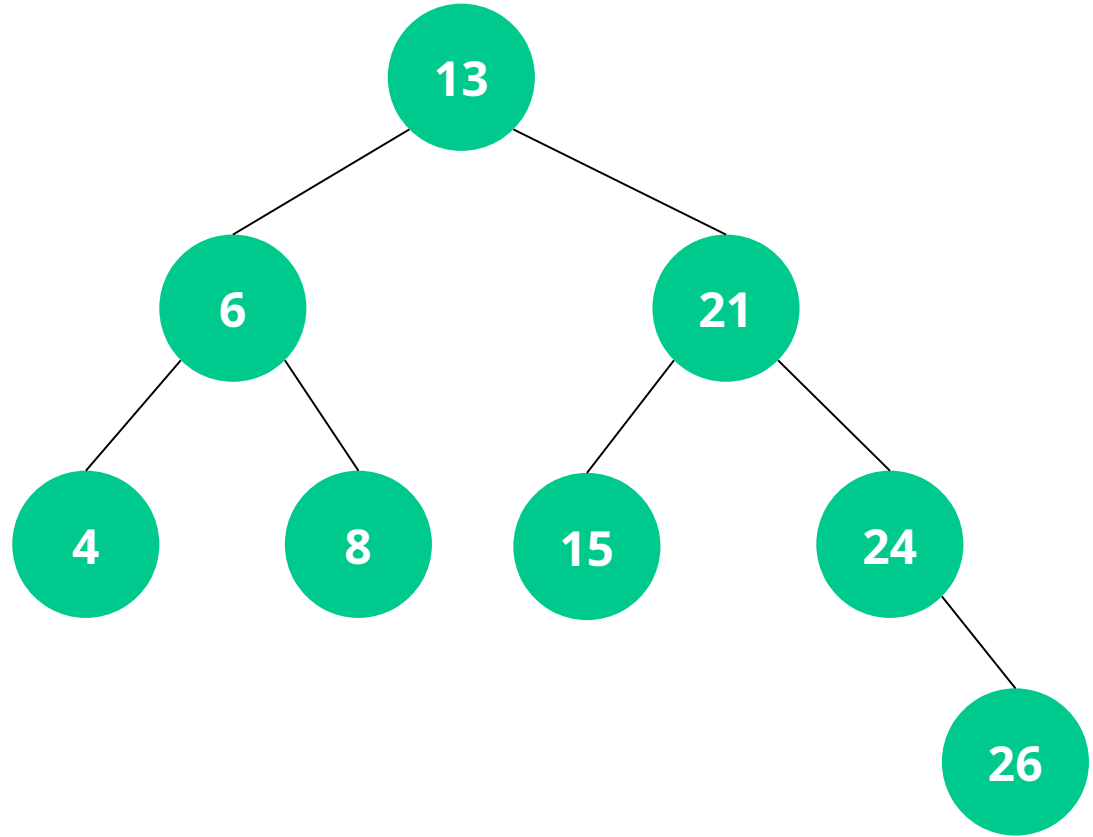
Left > **Root** > Right



Output:

Pre-order Traversal

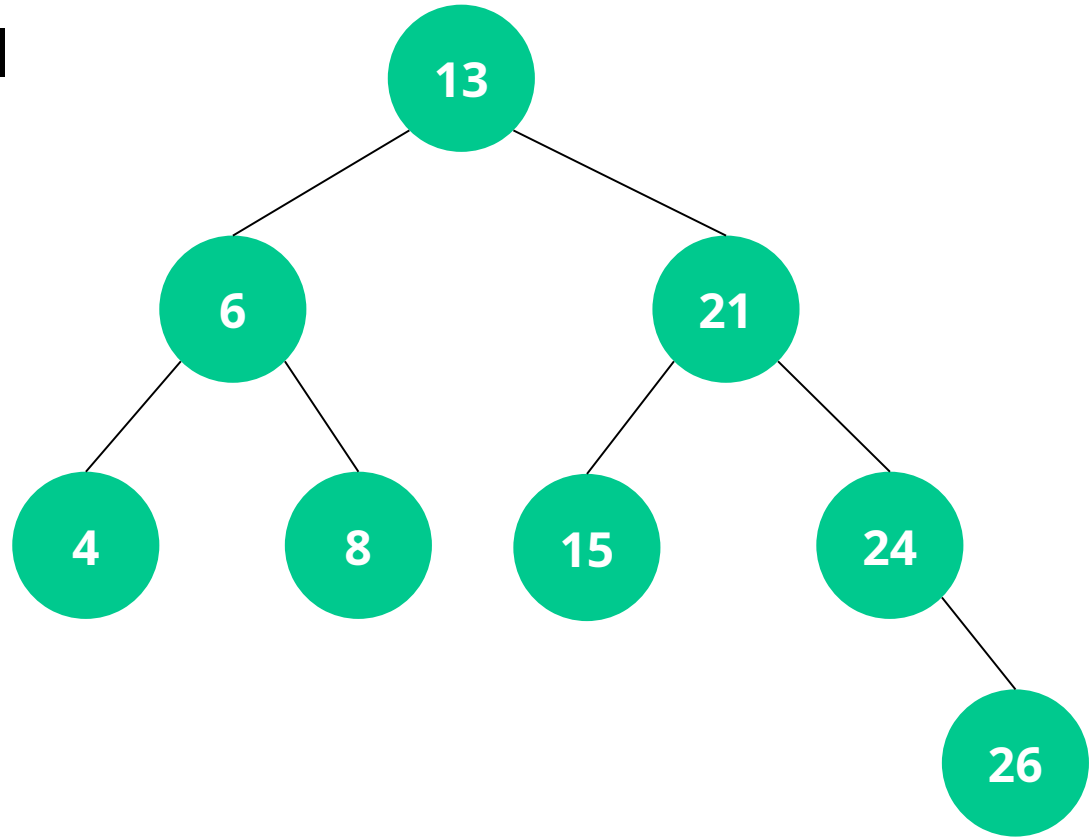
Root > Left > Right



Output:

Post-order Traversal

Left > Right > **Root**



Output:



Code Demo: In-order traversal

- ☐ Let's jump to VS Code for a code demo. We will highlight how **similar** all of the solutions are.

Break Time!

Take 5 mins to step away from the computer. Feel free to turn off your camera and return promptly after 5 mins.



10 minutes

AI Checkpoint #3



Discussion

2-3 minutes

Share an instance in which AI helped you:

- **debug a problem** 🐛
- **understand a new topic**



Discussion

2-3 minutes

Rewind to Checkpoint #1... have your answers changed?

What's the **easiest mistake** to make when using AI?

What's the **best tip** you know on how to make better use of AI?



Discussion

2-3 minutes

What skill would you next like to build using what you've learned in this class?

How do you plan to use **AI** to reach your goal?



60 minutes

Collaborative Problem Solving



Breakout Rooms

60 minutes

1. 🏃 Navigate to your breakout room.
2. 🙌 Introduce yourselves!
3. 🧑💻 Work on the problems for your selected unit.

10 minutes

Wrap Up



One Good Thing

Let's reflect on today's session. Maybe you got a question answered, overcame a specific coding challenge, or learned how to do something completely new!

In the chat, share at least one good thing you've learned or experienced during the session today.

Let's celebrate together!



Session Survey Time!





Final Reminders

- ☐ Finish the problem sets for additional practice
- ☐ Take the weekly HackerRank assessment by Sunday @ midnight
- ☐ Visit a **Study Hall Session** to deepen your understanding, work through challenges, and connect with course material alongside your peers!
- ☐ Peer <> Peer Mock Interviews are on Thursday
 - [Check out the Showrunner guide](#) for preparation!